

AI-Driven Mini Project

AI Knowledge Base Q&A System

End-to-End Requirement Document for React + Python/FastAPI + PostgreSQL Training

COMPLEXITY
Medium

BEST FOR
Strong fresher / AI interest

DURATION
10-12 working days

AI SCOPE
Ask questions from docs

What the fresher will build

A document knowledge base where users upload project documents and ask questions; AI answers using stored document content and shows source references.

Purpose

This document is written for college freshers. It explains the project like a real application, but keeps the scope small enough for one intern to build individually as an MVP.

Expected Output

- Working React UI with page-by-page screens
- FastAPI backend with clean APIs and validation
- PostgreSQL database with normalized tables
- One practical AI feature integrated through backend
- Demo flow with sample data

Recommended Training Rule

Keep the AI feature focused. Do not try to build an enterprise product. The fresher should first complete normal CRUD + screens, then add the AI flow as the main differentiator.

1. Project Overview

A document knowledge base where users upload project documents and ask questions; AI answers using stored document content and shows source references.

Short Real-Life Story

A new fresher joins a project and has many questions about setup, modules, and process. Instead of asking seniors repeatedly, the fresher opens the Knowledge Base, asks questions, and the AI answers from uploaded project documents with source snippets.

Who Will Use This Application?

- Admin/Document Owner
- Employee/User

What the Fresher Will Learn

- Build document library
- Implement text chunking/search
- Create chat-style UI
- Use AI for grounded answers
- Store Q&A history and feedback

Simple End-to-End User Journey



Suggested Technology Stack

- Frontend: React, React Router, Axios, reusable components, form validation
- Backend: Python FastAPI, Pydantic schemas, SQLAlchemy ORM, JWT authentication
- Database: PostgreSQL with normalized tables and foreign keys
- AI Integration: AI API called only from FastAPI service layer; API key stored in .env
- Deployment Demo: local frontend + backend + PostgreSQL is enough for intern training

2. Feature Scope

Core MVP Features

- Login and knowledge base dashboard
- Admin can upload/paste documents and add tags
- System stores document text chunks in PostgreSQL
- User can search documents by keyword
- User can ask question in chat screen
- AI answers using matching document chunks
- Show source document names/snippets
- User can mark answer helpful/not helpful

Extra Features - Add only after MVP is completed

- Unanswered question list
- Document versioning
- Category-wise KB
- Top asked questions
- Admin reprocess document

User Stories

Role	I want to...	So that...
Admin	I want to upload project documents	So users can learn from approved content
User	I want to ask questions	So I can find answers quickly
User	I want to see source references	So I can trust the answer
Admin	I want to see unanswered questions	So I can improve documents

Scope Control Note

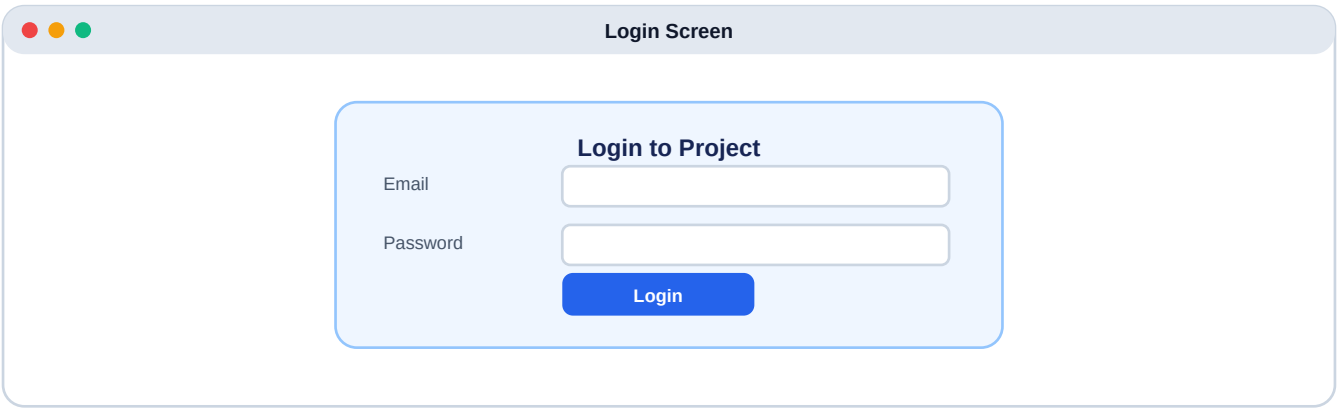
For intern training, the project should be completed as a working MVP first. Extra features should be treated as bonus tasks, not mandatory requirements.

3. Page-by-Page Screens

These screens show how a real user will use the application step by step. Freshers can use this section as the UI development guide.

Login Screen

Who uses this page?	Admin and user
Why this page is needed?	To access authorized knowledge base
What happens next?	User logs in and goes to KB dashboard.



Knowledge Base Dashboard Screen

Who uses this page?	Admin
Why this page is needed?	To monitor document library
What happens next?	Admin uploads or reviews documents.



Document Upload Screen

Who uses this page?	Admin
Why this page is needed?	To add knowledge content
What happens next?	Admin uploads text and processes it for Q&A.

Document Upload Screen

MENU

Dashboard

List

Create

Reports

Document Upload Screen

Document Title

Category

Tags

Document Text/File

Process

Save / Generate

AI Assistant

- Summarize document

- Create chunks

- Suggest tags

- Find key topics

Ask Knowledge Base Screen

Who uses this page?	User
Why this page is needed?	To ask questions from documents
What happens next?	User types question and receives grounded answer.

Ask Knowledge Base Screen

MENU

Dashboard

List

Create

Reports

Ask Knowledge Base Screen

Documents

Setup Guide

API Guide

User Manual

FAQ

Ask AI

User asks a question...

Page 5

Answer With Sources Screen

Who uses this page?	User
Why this page is needed?	To verify answer source
What happens next?	User marks answer helpful or asks follow-up.

Answer With Sources Screen

MENU

Dashboard

List

Create

Reports

Answer With Sources Screen

Record Details

Question:

AI answer:

Source document:

Source snippet:

Confidence:

AI Output / Activity

- Answer summary

- Referenced source

- Follow-up questions

- Helpful feedback

Q&A History Screen

Who uses this page?	Admin/user
Why this page is needed?	To review previous questions
What happens next?	Admin identifies common unanswered topics.

Q&A History Screen

MENU

Dashboard

List

Create

Reports

Q&A History Screen

Search

Document

Helpful

Date

Question	Document	Helpful	Asked By	Date
Sample 1	Sample 1	Sample 1	Sample 1	Sample 1
Sample 2	Sample 2	Sample 2	Sample 2	Sample 2
Sample 3	Sample 3	Sample 3	Sample 3	Sample 3

4. PostgreSQL Database Design

The fresher should create these tables with primary keys, foreign keys, created_at, updated_at, and proper indexes for frequently filtered columns.

Table	Important Fields
users	id, name, email, password_hash, role_id
documents	id, title, category, tags, uploaded_by, status, created_at
document_chunks	id, document_id, chunk_no, chunk_text, keyword_text, created_at
qa_sessions	id, user_id, title, created_at
qa_messages	id, session_id, question, answer, source_chunk_ids, helpful, created_at
document_processing_logs	id, document_id, status, error_message, processed_at

Suggested Database Rules

- Use integer or UUID primary keys consistently.
- Use foreign keys for user, role, status, and parent-child relationships.
- Add created_at and updated_at to all main tables.
- Do not store AI API keys in database.
- Store AI input/output logs only for learning/demo purpose. Avoid real confidential data.

5. FastAPI API Design

Below endpoints are enough for MVP. The intern can add more endpoints only after this basic flow works.

Method	Endpoint	Purpose
POST	/auth/login	Login
GET	/dashboard/kb	Knowledge base summary
POST	/documents	Upload/add document
GET	/documents	List documents
POST	/documents/{id}/process	Split document into chunks
POST	/qa/ask	Ask a question
GET	/qa/history	List Q&A history
PATCH	/qa/messages/{id}/feedback	Mark answer helpful/not helpful

Common API Response Format

```
{
  "success": true,
  "message": "Request completed successfully",
  "data": {}
}
```

Recommended API Rules

- Use Pydantic schemas for request and response validation.
- Use service layer for business logic and AI calls.
- Use repository/helper layer for database queries.
- Return clear error messages for validation failures.
- Add pagination for list APIs.

6. AI Integration Design

AI should be treated as one backend service. React should never call the AI provider directly. React calls FastAPI, and FastAPI calls the AI service.

AI Flow

- 1 Admin adds document text
- 2 Backend splits document into chunks
- 3 User asks question
- 4 Backend finds relevant chunks using keyword search or simple embeddings
- 5 AI answers only using selected chunks
- 6 Answer and source references are stored

Sample AI Prompt

Answer the user's question using only the provided document context. If the answer is not available in the context, say that the document does not contain enough information. Include a short answer and source title.

Expected AI Output Format

```
{
  "summary": "Short AI-generated summary",
  "result": "Structured result based on project need",
  "confidence_score": 0.85,
  "suggested_next_action": "Simple next action for the user"
}
```

AI Safety and Quality Rules

- Always validate AI response before saving it.
- Show AI output as suggestion, not final truth.
- Allow user to edit or confirm AI output.
- Handle AI timeout or failure with a friendly error message.
- Use sample/demo data only for training.

7. Frontend and Backend Structure

React Suggested Folder Structure

```
src/  
api/ # axios instance and API functions  
components/ # buttons, tables, forms, modals  
layouts/ # sidebar, header, protected layout  
pages/ # screen pages  
routes/ # React Router setup  
utils/ # helpers, constants, validation  
App.jsx  
main.jsx
```

FastAPI Suggested Folder Structure

```
app/  
main.py  
core/ # config, security, database connection  
models/ # SQLAlchemy models  
schemas/ # Pydantic request/response schemas  
routers/ # FastAPI route files  
services/ # business logic and AI service  
repositories/ # database query helpers  
utils/ # common helpers
```

Important React Components

- ProtectedRoute
- SidebarLayout
- DashboardCard
- DataTable
- FilterBar
- FormInput
- StatusBadge
- AIResultPanel
- ConfirmModal
- ToastMessage

Important Backend Services

- AuthService
- UserService
- MainModuleService
- AIService
- DashboardService
- ReportService

8. Business Rules and Validations

- Document title required
- Question required
- Do not answer without document context
- Show source references
- For intern MVP, PostgreSQL text search is enough; vector DB is optional

Common Validation Rules

- Required fields should be validated in React and FastAPI.
- Backend validation is mandatory even if frontend validates.
- User should see clear success and error messages.
- Unauthorized users should not access protected APIs.
- Soft delete can be used instead of permanent delete for important records.

Development Plan

Day	Task
Day 1	Understand requirement, finalize pages, design DB tables
Day 2	Create FastAPI project, database connection, models, migrations
Day 3	Implement login/JWT and basic protected routes
Day 4	Implement core CRUD APIs
Day 5	Create React layout, routing, login, dashboard
Day 6	Create list, form, detail screens and API integration
Day 7	Add AI API integration and store AI output
Day 8	Add filters, status flow, reports, and validations
Day 9	Testing, bug fixing, seed data, demo script
Day 10	Final demo and code review

9. Testing and Demo Checklist

Use this checklist before the fresher demo.

- | | |
|--|---|
| ☐ Login works with JWT | ☐ Dashboard shows real counts |
| ☐ Create/edit/view/list flow works | ☐ Search and filters work |
| ☐ AI feature works through backend | ☐ AI result is stored in database |
| ☐ Validation and error messages are clear | ☐ Status flow works where applicable |
| ☐ Code is clean and reusable | ☐ Demo can be explained end to end |

Demo Script

- 1 Login with sample user.
- 2 Show dashboard counts.
- 3 Create one new record using form screen.
- 4 Run AI feature and explain input/output.
- 5 Save AI output and show it in detail page.
- 6 Use search/filter in list page.
- 7 Update status or edit record where applicable.
- 8 Show report/history page.
- 9 Explain database tables and API flow.
- 10 Explain one challenge faced and how it was solved.

Final Review Criteria

Area	Expected Standard
UI	Clean layout, readable forms, working navigation
React	Reusable components, API integration, state handling
FastAPI	Clean routers, schemas, services, validations
PostgreSQL	Proper tables, relationships, sample seed data
AI	Backend AI integration, stored result, editable suggestion
Demo	Can explain end-to-end user journey